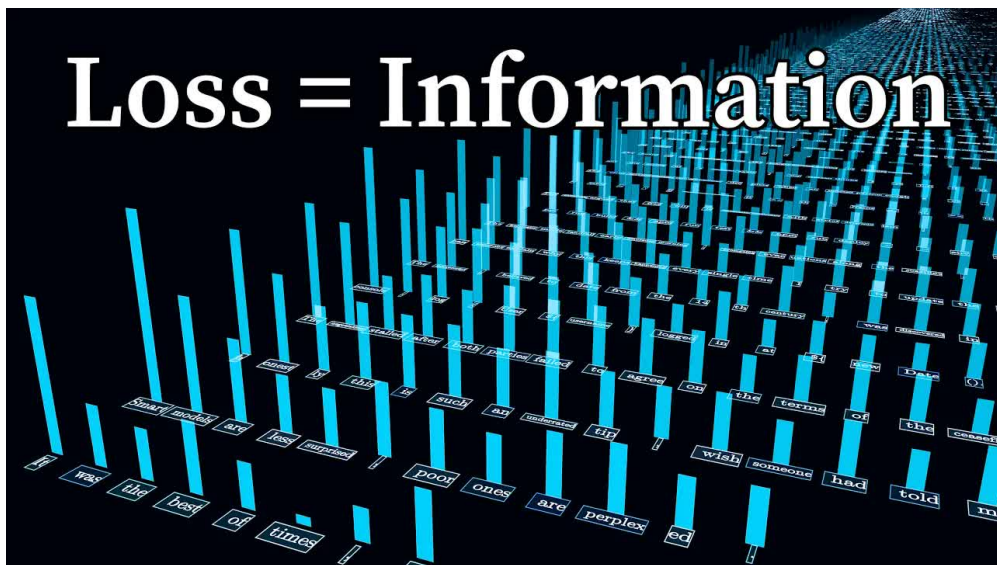


课程笔记：交叉熵是什么？——压缩即智能 (第二部分)

五道口纳什 & Codex

2026 年 8 月 20 日



视频作者/频道: 3Blue1Brown

发布日期: 2026-07-16

视频时长: 33:51

视频链接: <https://youtu.be/GlYgs6v2YfU>

PDF 制作 Skill: <https://github.com/wdkns/wdkns-skills>

目录

1 引言：压缩如何”听懂”语言	2
1.1 一个 2002 年的”野”论文：语言树与压缩	2
1.2 本视频的路线图	3
2 最优编码回顾：信息量与比特	3
2.1 机器人指令的玩具模型	3
2.2 信息内容与最优编码长度	4
3 交叉熵的定义：当编码遇上”另一个世界”	5
3.1 分布变了，编码没变	5
3.2 公式化：交叉熵的符号与图示	7
4 直觉与例子：两个事件的情形	8
4.1 例子 1：均匀 Q 与偏斜 P	8
4.2 例子 2：偏斜 Q 与均匀 P——顺序很重要	9
4.3 最小值性质：交叉熵的关键属性	9
5 交叉熵之于语言树：开头的例子圆回来了	11
6 预训练语言模型：交叉熵作为损失函数	12
6.1 从压缩跳到机器学习	12
6.2 损失 = 模型视角下的平均信息量	13
6.3 预训练的全部内容	14
6.4 为什么叫”交叉熵损失”？	15
7 为什么损失必须是负对数：约束优化的必然	16
7.1 名字的例子	16
7.2 反过来：约束优化强迫你使用对数	17
8 蒸馏：交叉熵公式更”显式”的用法	18
8.1 设置	18
9 总结与延伸：从压缩到智能	19
9.1 把两条线索织在一起	19
9.2 KL 散度：交叉熵 - 熵	21
9.3 笔记人的综合思考	24
9.4 拓展阅读	24

1 引言：压缩如何”听懂”语言

本视频是 3Blue1Brown 《Compression is Intelligence》（压缩即智能）系列的第二部分，核心目标只有一个：把交叉熵（cross-entropy）这个公式讲透。这个公式在机器学习里几乎无处不在——语言模型的训练损失、蒸馏、KL 散度，全都以它为基础。而 3Blue1Brown 的独特视角是：交叉熵并不只是深度学习工程师的”工作工具”，它首先是信息论中关于压缩的一个自然结论。

系列背景

第一部分（Compression is Intelligence Part 1）建立了”信息内容 $= -\log_2 p$ ”这一核心概念：一个事件越不可能发生，它携带的信息量越大。本视频从那里继续，先补上压缩与交叉熵的动机，再转到语言模型训练。

1.1 一个 2002 年的”野”论文：语言树与压缩

视频从一个令人惊讶的例子出发。2002 年有一篇论文 *Language Trees and Zipping*（语言树与压缩），作者 Dario Benedetto、Emanuele Caglioti 和 Vittorio Loreto（罗马大学）。它做的事情在今天看来仍然很惊人：

1. 给你一堆用不同语言写成的文本文档；
2. 你的目标是自动把它们按语言聚类（cluster）；
3. 更进一步，还希望恢复出这些语言之间的”亲缘关系树”——即哪些语言共享更近的共同祖先；
4. 而唯一的工具，是编写一个处理文本的函数——没有预先内置任何语言学知识，也没有现代语言模型（那是 2002 年）。

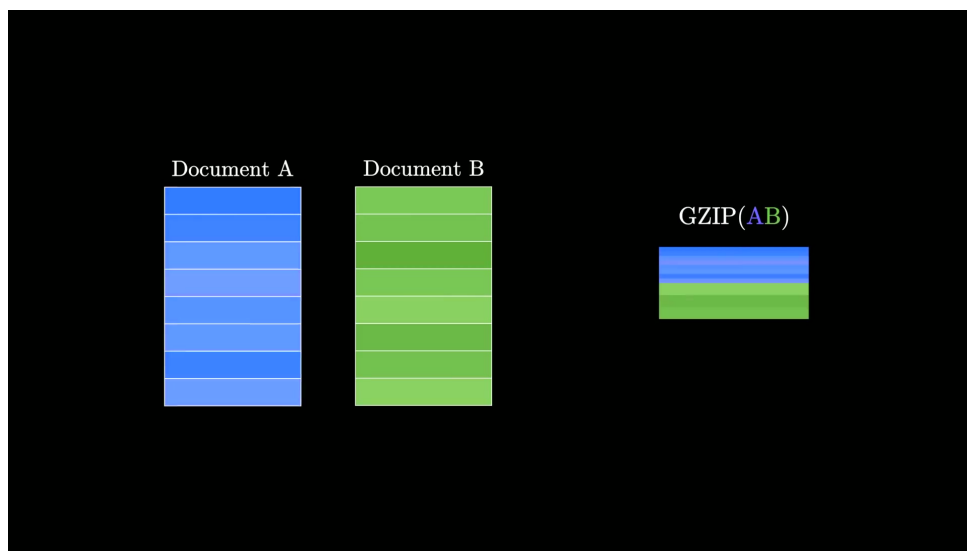


图 1: 论文的核心想法：把文档 A 和文档 B 的一小段拼接起来，再用 gzip 压缩。¹

答案居然是 gzip——就是你在电脑上用来把文件压缩变小的那个工具。基本想法如下：

1. 给定两份文档 A 和 B，把 B 的一小段拼接到 A 的末尾；

¹视频画面时间区间：00:00:50–00:01:00。

2. 压缩拼接结果 $GZIP(AB)$;
3. 把它与单独压缩 A 的文件大小 $GZIP(A)$ 比较;
4. 两者之差衡量的是: 当压缩器主要“熟悉” A 的模式时, B 的那一小段被压缩得有多好。 B 与 A 的语言模式越相似, 压缩效果越好, 差值越小。

核心思想

“共同压缩得好” $\approx$ “语言模式相近”。作者用这个纯粹基于压缩的距离度量, 就恢复了印欧语系的语言谱系树——这是理论(信息论)与工程(gzip 实现)服务于“看似属于机器学习”的任务的绝佳范例。

更有趣的是, 同样的技巧还能用于判断作者身份(authorship attribution): 文档与某位作者的文本压缩得越紧, 越可能是他写的。这为后面“交叉熵是衡量两个环境模式差异的工具”埋下了伏笔。

1.2 本视频的路线图

1. 前半部分: 交叉熵是什么, 它如何从压缩问题中自然涌现, 以及如何可视化;
2. 后半部分: 预训练(pre-training)与蒸馏(distillation)语言模型的基础;
3. 关键点: 交叉熵在后半部分的出现方式看起来与压缩毫无关系, 但同一个公式出现在两个不同场景, 是数学在向你“眨眼”(wink), 暗示着深层联系;
4. 结尾: 用压缩的视角重新审视语言模型训练——不是“预测下一个 token”, 而是“压缩”。

2 最优编码回顾: 信息量与比特

2.1 机器人指令的玩具模型

交叉熵最好在“把消息编码成比特串”的语境中引入。这是第一部分的延续: 把消息看作符号序列(比如文本的字符), 每个新符号都像是从某个概率分布中采样得到的, 这些概率告诉你压缩的极限。

自然语言极其复杂, 所以视频沿用第一部分那个简化玩具模型: 向远方机器人发送四种指令——上(up)、下(down)、左(left)、右(right), 每个符号从如下分布中采样:

$$P(\text{up}) = \frac{1}{2}, \quad P(\text{down}) = \frac{1}{4}, \quad P(\text{left}) = P(\text{right}) = \frac{1}{8}$$

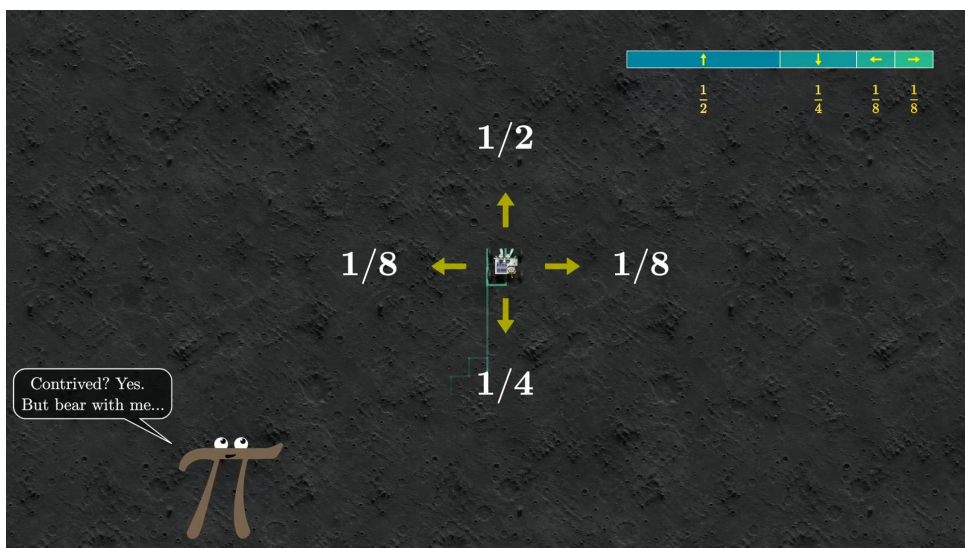


图 2: 旧分布: up 占 $1/2$, down 占 $1/4$, left 和 right 各占 $1/8$ 。3Blue1Brown 特意标注“Contrived? Yes. But bear with me.” (刻意构造? 是的, 但请忍耐)。²

第一部分得到的结论是: 这个分布的最优编码把 up 编成 1 个比特, down 编成 2 个比特, left 和 right 各编成 3 个比特。



图 3: 针对旧分布的最优编码: up $\rightarrow$ 1 bit, down $\rightarrow$ 2 bits, left/right $\rightarrow$ 3 bits。³

2.2 信息内容与最优编码长度

这个玩具例子的意义在于它体现了一个更普遍的事实: 对于最优编码, 分配给某个符号的比特数恰好是

$$L_i = -\log_2 p_i$$

²视频画面时间区间: 00:03:50–00:04:00。

³视频画面时间区间: 00:04:08–00:04:16。

其中 p_i 是该符号出现的概率。这个负对数表达式需要一点时间适应——3Blue1Brown 调侃说，他希望历史能把它命名为“以 $1/2$ 为底的对数” ($\log_{1/2} p$)，因为它实际上问的是“要把某物减半多少次才能降到某个量”。

香农的信息内容定义

Claude Shannon 把这个表达式定义为事件的**信息内容** (information content): $I(p) = -\log_2 p$ 。事件越稀有，信息量越大。

当然，通常概率不会是那么整齐的数，信息值往往不是整数，所以“最优码”不能简单到“每个符号配一个固定比特串”。更一般的思考方式是：编码整条消息时，整条消息的信息内容等于所有符号信息内容之和，而消息的最优编码比特数近似等于该总信息量。因此，分数信息量在这里有非常实在的意义。

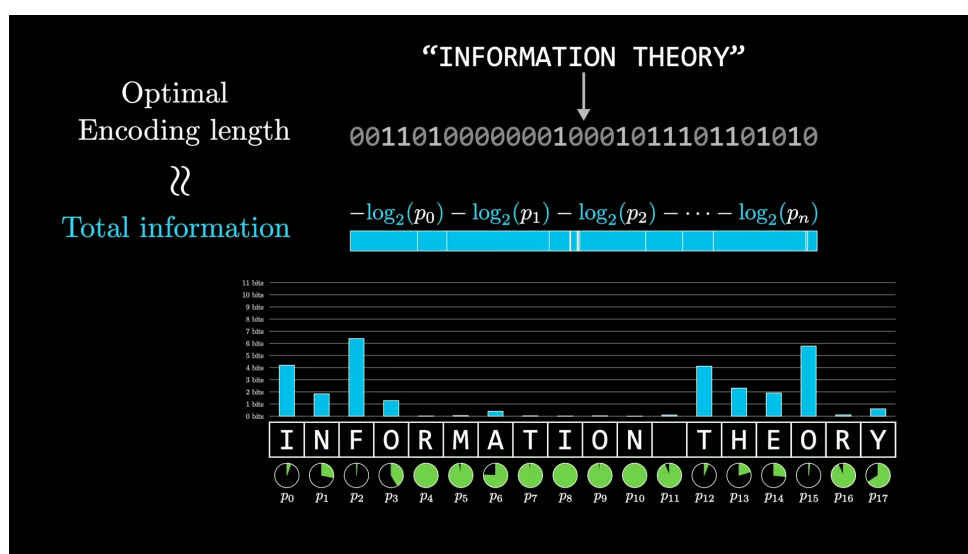


图 4：最优编码长度与信息内容：例如“INFORMATION THEORY”这条消息，最优编码总长度 $\approx \sum_i -\log_2 p_i$ 。⁴

常见误解

不要以为“最优编码”就是给每个符号分配一个固定长度的二进制串。信息值通常不是整数，真正的含义是：在极限意义上，一个消息的最优编码长度趋近于其总信息内容 $\sum_i -\log_2 p_i$ 。分数比特在平均意义上是有意义的。

3 交叉熵的定义：当编码遇上“另一个世界”

3.1 分布变了，编码没变

现在进入第二部分的核心场景。假设航天局改变了计划，希望机器人整体更偏右移动，于是新的符号分布变为：

$$P(\text{up}) = P(\text{down}) = \frac{1}{8}, \quad P(\text{left}) = \frac{1}{4}, \quad P(\text{right}) = \frac{1}{2}$$

⁴视频画面时间区间：00:05:00–00:05:16。

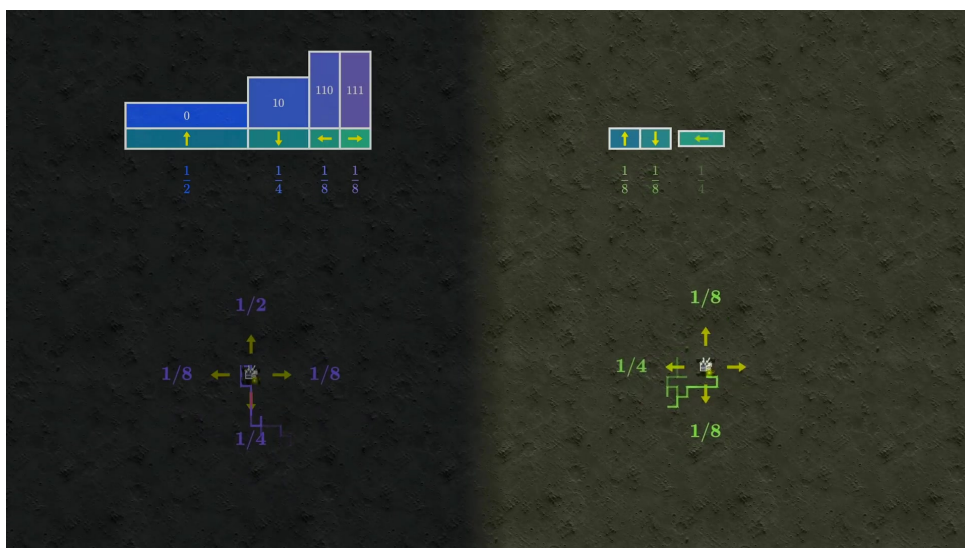


图 5: 新分布: up 和 down 各占 1/8, left 占 1/4, right 占 1/2。⁵

但是——假设你的编码和解码系统是硬编码的，仍然使用针对旧分布优化的那套编码方案。这显然低效了，但究竟低效多少？

计算并不难：单比特指令 up 现在只出现 1/8 的时间，两比特的 down 也只出现 1/8，另外两个指令各需 3 比特，合起来占了 75% 的时间。把它们加权求和：

$$\frac{1}{8} \cdot 1 + \frac{1}{8} \cdot 2 + \frac{1}{4} \cdot 3 + \frac{1}{2} \cdot 3 = 0.125 + 0.25 + 0.75 + 1.5 = 2.625 \text{ bits/指令}$$

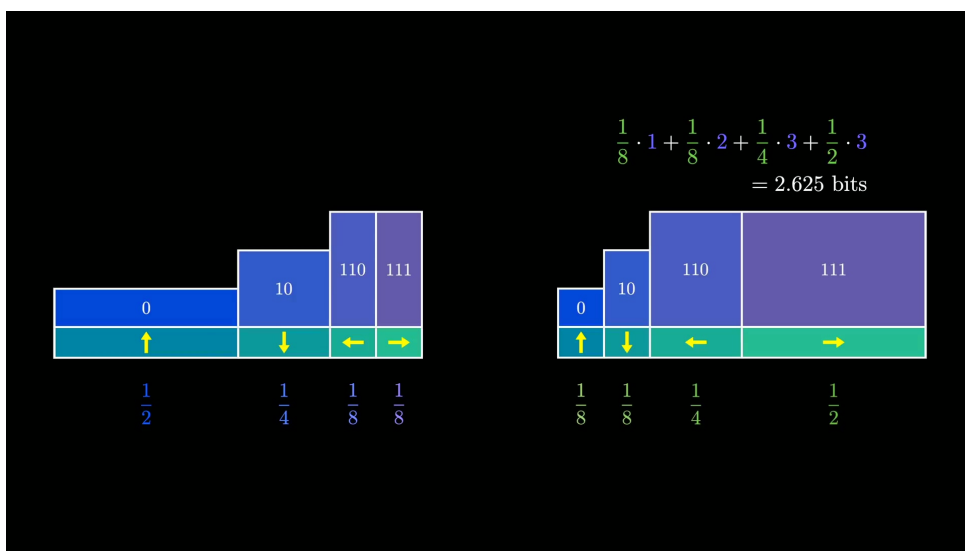


图 6: 用旧编码传输新分布下的消息，平均每条指令消耗 2.625 比特。⁶

⁵视频画面时间区间：00:05:24–00:05:36。

⁶视频画面时间区间：00:05:52–00:06:12。

交叉熵的非正式定义

这个数字——”针对一个环境优化的压缩方案，在另一个环境下平均每条消息消耗的比特数”——就是**分布 Q 相对于分布 P 的交叉熵**。用语言来说：你在一个环境下优化的压缩方案，在另一个环境下表现如何？

如果你此刻想起开头那个 zipping 和语言树的例子，那就对了——这两件事的关联马上就要浮出水面。

3.2 公式化：交叉熵的符号与图示

把新分布的概率统一记为 $P(p_1, p_2, \dots, p_n)$ ，把旧分布（编码为之优化的分布）记为 $Q(q_1, q_2, \dots, q_n)$ 。不再局限于四种机器人指令，考虑一般的符号序列——可以是四个指令，可以是英文字符，也可以是任何你想象的东西。

针对分布 Q 优化的编码，分配给第 i 个符号的比特数是 $-\log_2 q_i$ 。当实际出现的分布是 P 时，平均每条消息的比特数是一个加权和：对每个符号，取其出现比例（概率 p_i ）乘以该符号在 Q 编码下的信息内容 $-\log_2 q_i$ ：

$$\text{交叉熵} = \sum_i p_i (-\log_2 q_i)$$

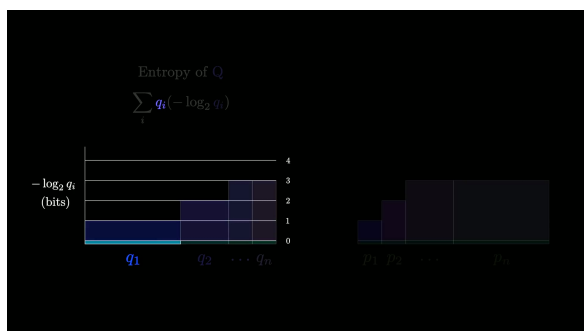
注意：两个分布的”分工”完全不同

- 权重（乘在括号外的 p_i ）来自**实际环境 P**——它决定每个符号出现的频率；
- 码长（括号内的 $-\log_2 q_i$ ）来自**编码所优化的分布 Q**——它决定编码方案。
- 所以交叉熵的简写记号在不同文献里五花八门（ $H_P(Q)$ 、 $H(Q; P)$ 等），关键是记住这个求和本身。

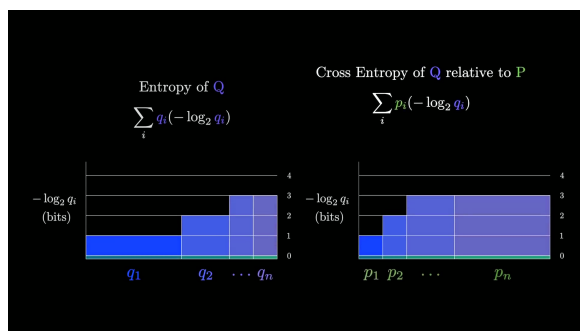
作为对比，如果实际分布就是 Q 本身，那么平均比特数是熵（entropy）：

$$H(Q) = \sum_i q_i (-\log_2 q_i)$$

可视化上，熵就是柱状图面积：每根柱子的宽度是 q_i ，高度是 $-\log_2 q_i$ ；而交叉熵的图里，宽度换成 p_i ，高度仍是 $-\log_2 q_i$ 。



(a) 熵 $H(Q) = \sum_i q_i(-\log_2 q_i)$: 宽度为 q_i , 高度为 $-\log_2 q_i$ 。



(b) 交叉熵 $\sum_i p_i(-\log_2 q_i)$: 宽度换成 p_i , 高度仍是 $-\log_2 q_i$ 。

图 7: 熵与交叉熵的柱状图对比。⁷

4 直觉与例子：两个事件的情形

4.1 例子 1：均匀 Q 与偏斜 P

为了建立直觉，考虑只有两个事件的分佈。设 Q 是均匀分佈（每个事件 50%）， P 更偏斜（90% 与 10%）。

从 Q 的视角看，每个事件有 1 比特信息，针对该分佈的最优码每个符号只用 1 比特，所以 $H(Q) = 1$ bit。

交叉熵呢？如果这套编码被用于新的符号分佈，平均每符号多少比特？在这个例子里，因为每个符号的信息量相同（都是 1 比特），权重不起作用，交叉熵也是 1 bit。

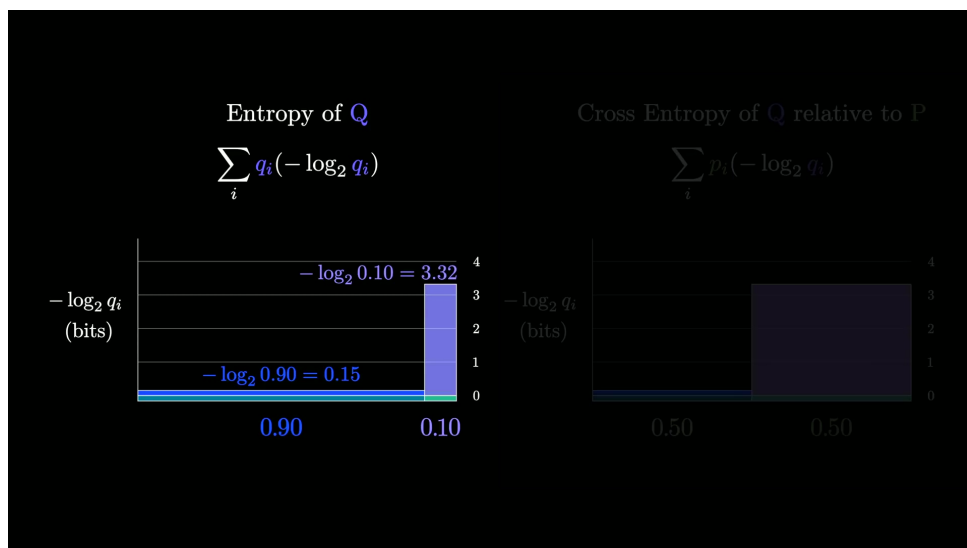


图 8: 偏斜情形： $-\log_2 0.10 = 3.32$ ——罕见事件的信息量很大。⁸

⁷ 视频画面时间区间：00:07:30–00:08:26（熵图 00:07:30–00:07:38，交叉熵图 00:08:00–00:08:26）。

⁸ 视频画面时间区间：00:09:38–00:09:46。

4.2 例子 2：偏斜 Q 与均匀 P——顺序很重要

现在把角色对调：Q 是偏斜分布（90%–10%），P 是均匀分布（50%–50%）。

从 Q 的视角看，那个很常见的事件信息量极小，另一个事件信息量很大。所以针对 Q 的最优码，对常见事件应该远少于 1 比特（平均而言），代价是罕见事件平均花多个比特。算下来 $H(Q) < 1$ bit。

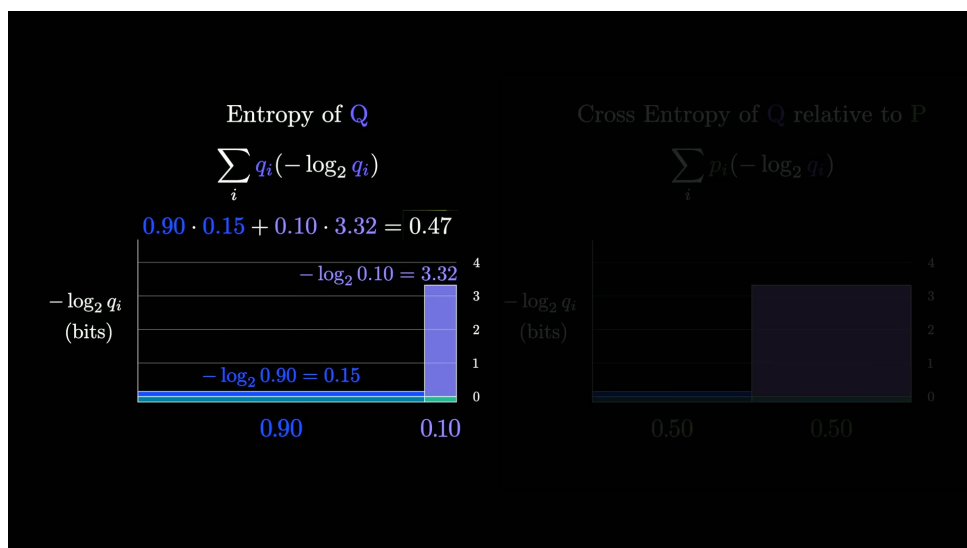


图 9: 熵的计算： $0.90 \times 0.15 + 0.10 \times 3.32 = 0.47$ bits。⁹

但交叉熵呢？针对偏斜分布优化的编码，面对两个事件等可能的新现实时，劣势就显现了：加权和约为 **1.74 bits**。

P 和 Q 的角色不对称

调换 P 和 Q 的角色，交叉熵的值会改变。这符合直觉：在公式里 P 和 Q 的作用完全不同（一个当权重，一个定码长）。

4.3 最小值性质：交叉熵的关键属性

只有两个事件时，Q 只依赖一个自由参数 q_1 （补集锁定为 $1 - q_1$ ），P 同理只依赖 p_1 。自由参数这么少，就可以画图了，而这会凸显一个关键性质。

固定 P，把交叉熵当作 Q 的函数画出来：图像达到一个非常清晰的最小值，而且取到最小值的位置正是 P 与 Q 完全相同的地方。

⁹视频画面时间区间：00:09:56–00:10:08。

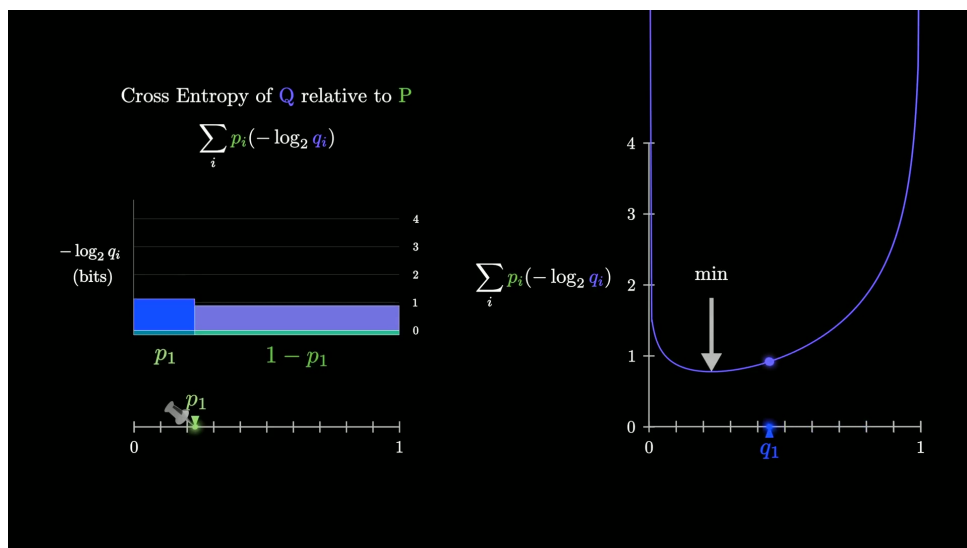


图 10: 固定 P , 把交叉熵画成 Q 的函数: 最小值出现在 $Q = P$ 处。¹⁰

这完全合理: 交叉熵问的问题是”为一种环境优化的编码在另一种环境表现如何”, 当两个环境一致时压缩效率当然最高。

还可以更有趣: 移动 P 的值, 整个图像形状改变, 同时追踪图像最小值点的轨迹——画出一条新曲线 (绿色)。这条曲线代表什么? 最小值交叉熵是”现实服从分布 P 时最高效的压缩”, 这正是 P 的熵。在二事件情形下, 绿色曲线就是熵 $H(P)$ 作为 p_1 的函数。

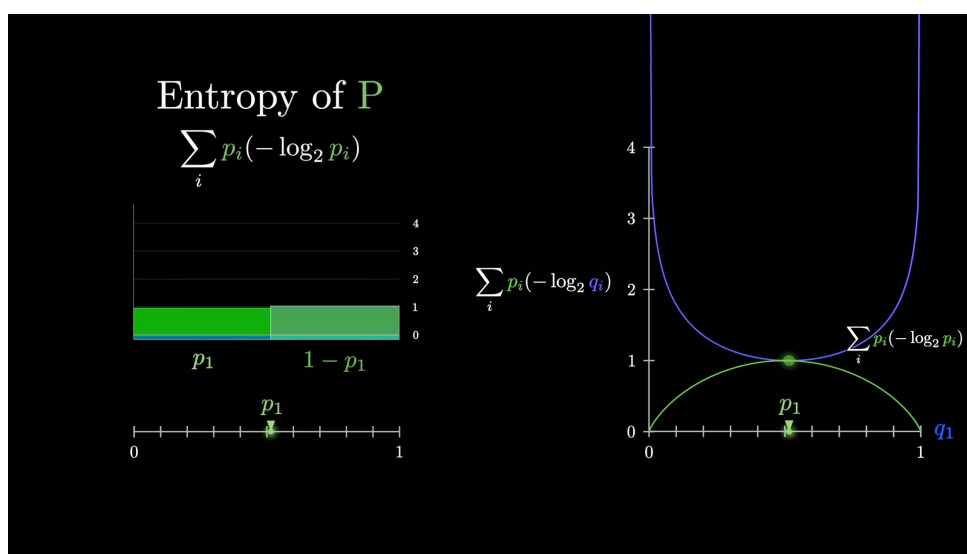


图 11: 绿色曲线: 随着 P 移动, 交叉熵最小值点的轨迹就是 $H(P)$ ——熵函数。¹¹

本视频最该记住的一条

把 P 固定、 Q 当变量: 交叉熵在 $Q = P$ 时取到**最小值**, 且最小值恰好等于 P 的熵 $H(P)$ 。

多事件的一般情形更难可视化, 但可以在脑子里这样保持这个事实: 保持柱状图宽度 (P) 固定,

¹⁰ 视频画面时间区间: 00:10:50–00:11:02。

¹¹ 视频画面时间区间: 00:11:28–00:11:56。

让 Q （高度，即 $-\log_2 q_i$ ）变化——这个面积有一个下界，在 $Q = P$ 时达到，下界值就是 $H(P)$ 。

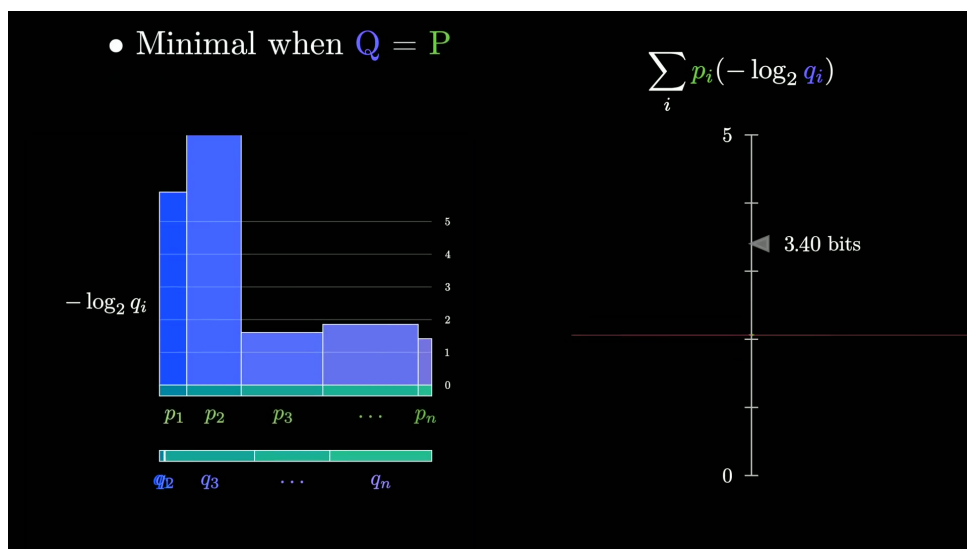


图 12: 柱状图视角：宽度固定为 P ，高度随 Q 变化，面积在 $Q = P$ 时最小。¹²

5 交叉熵之于语言树：开头的例子圆回来了

在解释为什么这个关键性质让交叉熵如此适合训练大语言模型之前，值得停下来想一想：开头的 zipping 与语言树例子，本质上就是交叉熵的应用。

回顾关键想法：要衡量两份文本文档 A 和 B 有多不同，取 B 的一小段拼到 A 上再压缩，比较拼接压缩后的大小与单独压缩 A 的大小。因为第一次压缩主要针对 A 的模式优化，这个差值问的正是交叉熵问的问题：为一种环境优化的压缩方案，面对另一种环境表现如何？

严谨性说明

作者定义的具体距离度量比这个简单差值复杂一些；而且严格说这并不等于交叉熵：其一，它不是在比较两个概率分布，而是把一份文档当作某个分布的样本，因此差值更像是交叉熵的**经验估计**；其二，gzip 不是达到香农极限的完美压缩（远未达到），它靠的是找到文本序列的重复并把后面的实例替换成指向前面的指针。但作者的想法是：只要它近似交叉熵，就能得到有用的距离度量，从而完成真正有趣的 NLP 任务。

交叉熵的普遍用途

交叉熵在实践中的一个常见主题：当你想要量化“一种环境中的模式与另一种环境中的模式有多不同”时，它就会出现。语言树是“一种语言 vs 另一种语言”，而今天的主要应用是“模型对语言的理解 vs 训练数据所代表的真实语言模式”。

¹²视频画面时间区间：00:12:31–00:12:46。

6 预训练语言模型：交叉熵作为损失函数

6.1 从压缩跳到机器学习

视频中闪现的动画展示的是 transformer 的底层架构（由 3Blue1Brown 的朋友 Clayton Rabideau 制作，NanoGPTAnimation）。但接下来采用更简化的示意图：一个由大量可调参数控制的”黑箱”。

语言模型基础回顾

- 现代语言模型把文本切分为 token（通常是词或词的一部分）；
- 语言模型是一个函数：输入任意 token 序列，输出所有可能 token 上的概率分布；
- 这个输出分布就是它”预测下一个 token”的分布；
- 训练需要损失函数：量化预测总体上是好是坏。

一个宽松的直觉是：损失函数的图像像一座山，训练就是在山上寻找最低点。但任何这样的图像都必须打上大大的问号——它展示的是只有两个输入（ x 和 y ）的函数，而真正的损失函数输入是语言模型的数十亿参数，真实的图像得住在数十亿维空间里。尽管如此，核心思想仍然成立：训练就是自动地调整参数、沿着山坡下坡、不断降低损失。

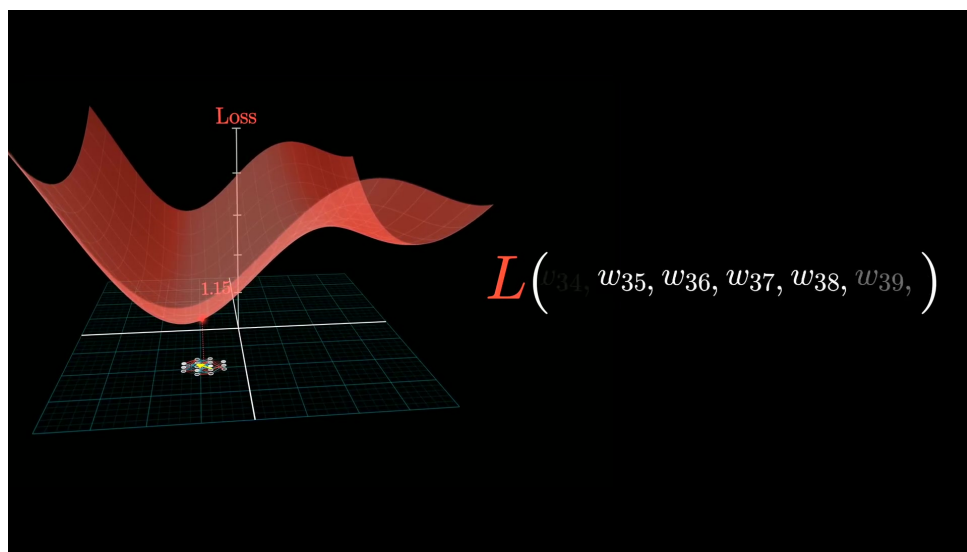


图 13: 损失函数的示意”山坡”： $L(w_{34}, w_{35}, \dots)$ 。真实损失函数有数十亿维，但”下山”的思想不变。¹³

自动调整参数的过程是梯度下降（gradient descent）与反向传播（backpropagation）。对工程师而言，一旦定义好损失函数，主要工作就完成了——深度学习标准机制会接手。损失函数针对一大堆训练数据定义，在语言模型语境下就是海量文本样本（通常来自互联网）。

¹³ 视频画面时间区间：00:16:26–00:16:32。

关键设计问题

你要写下什么样的数学函数来描述模型在所有数据上的表现，使得”最小化这个函数”恰好等价于”提升模型性能”？

6.2 损失 = 模型视角下的平均信息量

一个非常好的思考方式：损失函数衡量的是从模型视角看，每个 token 的平均信息量。具体来说：

1. 对训练数据中的一段文本，计算模型给其中每个新 token 的概率；
2. 对每个前缀子序列，让模型预测下一个 token，然后看真实出现的下一个 token（训练示例里实际存在的那个），记录模型给它的概率；
3. 损失 = 这些概率的负对数之和的平均（跨所有 token、所有示例）。

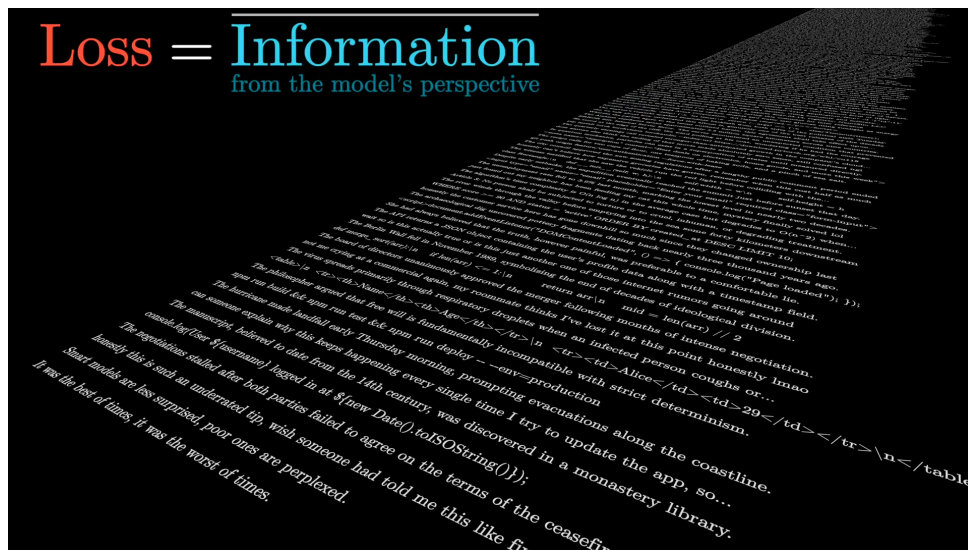


图 14: 每个 token 下方的小饼图表示模型分配给”真实下一个 token”的概率；损失取这些概率的负对数。¹⁴

为什么是负对数？如前面多次讨论的，它可以理解为”事件有多令人惊讶”。聪明的模型真正跟上了内容，就会对看到的东西不惊讶——给真实下一个 token 更高的概率，所有信息值都更低；而随机或训练不佳的模型会一直困惑，给所有东西低概率、时刻惊讶。

¹⁴ 视频画面时间区间：00:17:38–00:18:06。

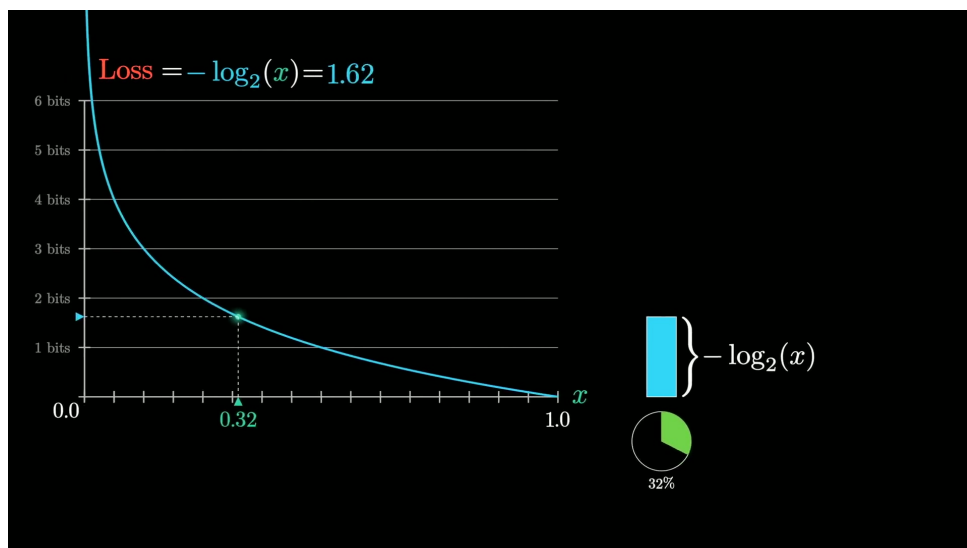


图 15: 负对数曲线作为损失: 给真实 token 赋低概率会被严厉惩罚, 概率高则损失很小。¹⁵

为什么是自然对数?

机器学习中几乎总是用自然对数而非以 2 为底的对数。这不影响本质——两者只差一个常数因子, 而常数因子会被“学习率”吸收。动机纯粹是自然对数求导更干净, 而求导正是梯度下降的核心。

6.3 预训练的全部内容

从某种意义上说, 这就是预训练的全部:

预训练损失

遍历每个 token, 取模型给它的概率的负对数, 然后对所有 token 求平均——不是某个示例里的所有 token, 而是整个庞大训练集的所有 token (希望它代表了你想建模的语言的显著样本)。用深度学习机制最小化这个损失, 让它趋近于语言的熵, 剩下的就是一个强大的通用预测器。

¹⁵ 视频画面时间区间: 00:18:45–00:19:04。

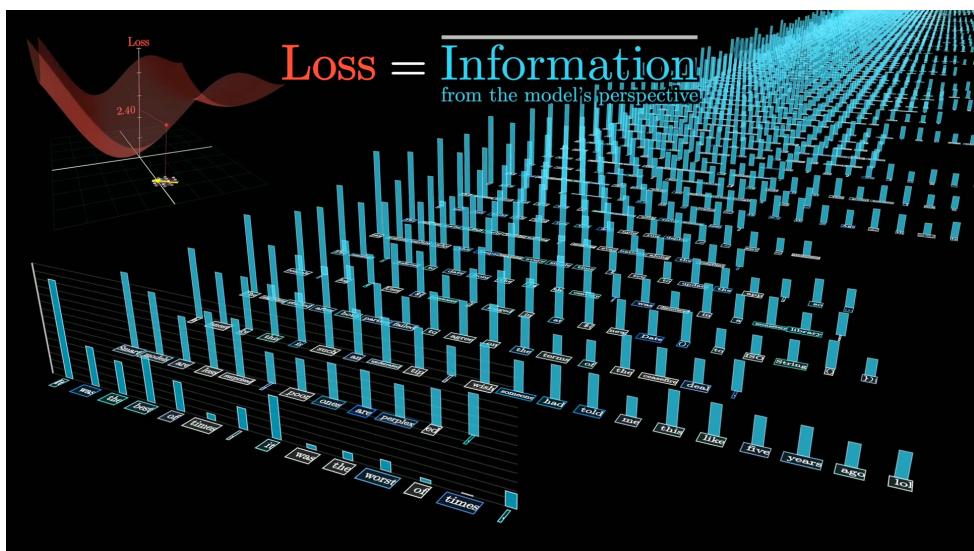


图 16: 预训练损失随训练下降：智能模型普遍”不惊讶”，随机模型总是”困惑”。¹⁶

当然，实际远不止这些——批处理 (batching)、特定的优化器，以及在实践中达到惊人规模本身是一项完整的工程壮举。但仅从损失函数的抽象层面看，它惊人地简单：平均每个 token 的信息量。

6.4 为什么叫”交叉熵损失”？

你可能会想，这个损失函数应该叫”信息损失”或”对数损失” (log loss)，但人们叫它**交叉熵损失** (cross-entropy loss) ——乍一听很奇怪，因为到目前为止的解释里根本没有用到交叉熵公式。

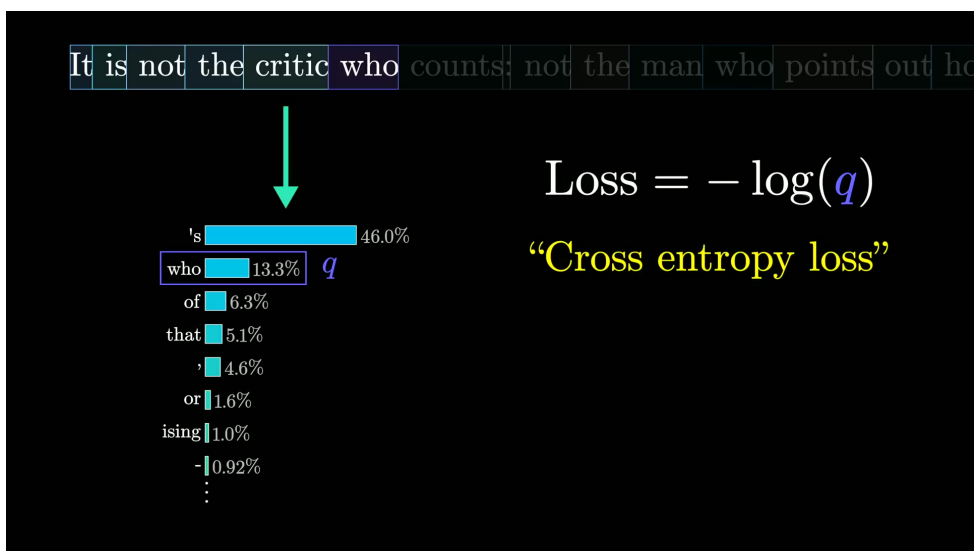


图 17: 常见的（令人不满意的）解释方式：交叉熵是”模型输出”与”把全部概率质量集中到唯一真实下一个 token 的完全偏斜分布”之间的交叉熵。¹⁷

有一种非常令人不满意的常见解释：说我们是在计算”模型对某 token 的输出”与”一个完全偏斜分布（所有概率质量集中在唯一真实下一个 token 上）”之间的交叉熵。套用公式，右边那一堆

¹⁶ 视频画面时间区间：00:19:45–00:20:10（此图为 00:19:50–00:20:06 附近的损失下降画面）。

¹⁷ 视频画面时间区间：00:20:38–00:20:50。

权重全是 0，把大部分项消掉，剩下的只是模型给真实下一个 token 的概率的负对数。

这种解释为什么”lame”（无聊）

3Blue1Brown 认为这种解释没有价值：它完全没有解释为什么一开始要用交叉熵；如果一切最终都消掉了，为什么不直接叫它 log loss 或 information loss？

他更满意的解释是：先说明为什么选择对数”根本不是选择”——你的手几乎是被迫的；当你看到这一点时，交叉熵公式就自然跳出来了。

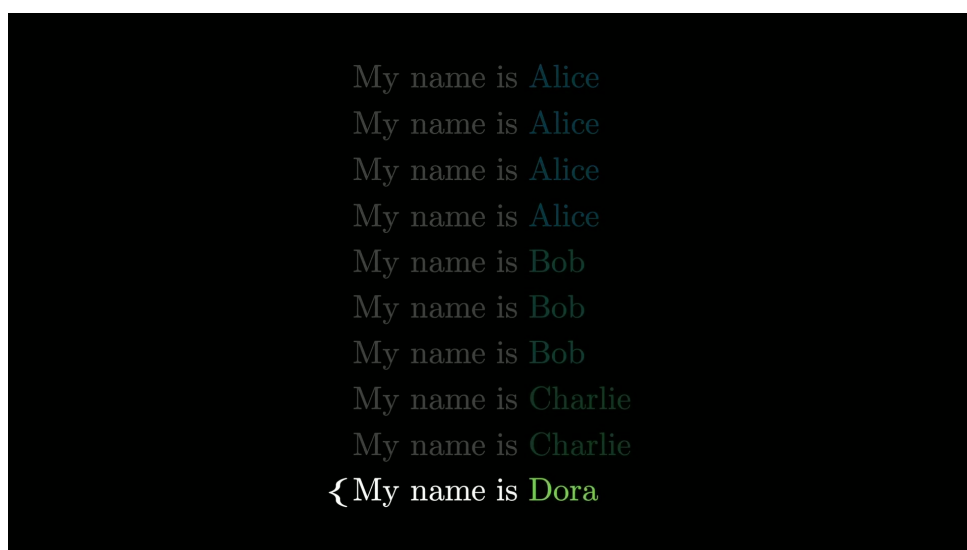
7 为什么损失必须是负对数：约束优化的必然

7.1 名字的例子

到目前为止只关注单个示例、单个 token 预测。我们把模型给那个 token 的概率变成某种损失值，告诉模型”应该多难受、多积极地修正自己”。

起初看起来唯一的约束是这个函数应该是递减的：低概率高损失，接近 1 的概率低损失。给定无穷多具有这种粗略形状的函数，什么使某一个更好？把未知函数记为 F ，寻找我们想要的额外性质。

设想”My name is _____”这样的模式。它在预训练数据里会出现很多次：常见的名字出现多次，罕见的名字出现较少。当输入”My name is”时，模型在所有可能的下一 token 上产生一个分布——把模型输出分布的概率记为 Q 。



My name is Alice
My name is Alice
My name is Alice
My name is Alice
My name is Bob
My name is Bob
My name is Bob
My name is Charlie
My name is Charlie
{ My name is Dora

图 18: ”My name is Alice” 在训练数据中出现多次；常见名字出现频率高，罕见名字低。¹⁸

问题是：与这个预测相关的总损失是多少？如果只看该模式在训练数据中出现的一个实例，损失是 $F(q)$ （无论 F 是什么）。但关键在于，这只是与该示例相关的总损失的一部分：该名字的每个其他实例都贡献同样的量，每个其他名字的每个实例也贡献类似的量——只是输入的是模型给那些名字的概率。

用 P 表示某个名字在训练数据中出现的比例。那么模型输出在所有出现该模式的示例上的平均损失是一个以 P 为权重的加权平均：

¹⁸ 视频画面时间区间：00:22:34–00:22:50。

$$\text{平均损失} = \sum_i p_i \cdot F(q_i)$$

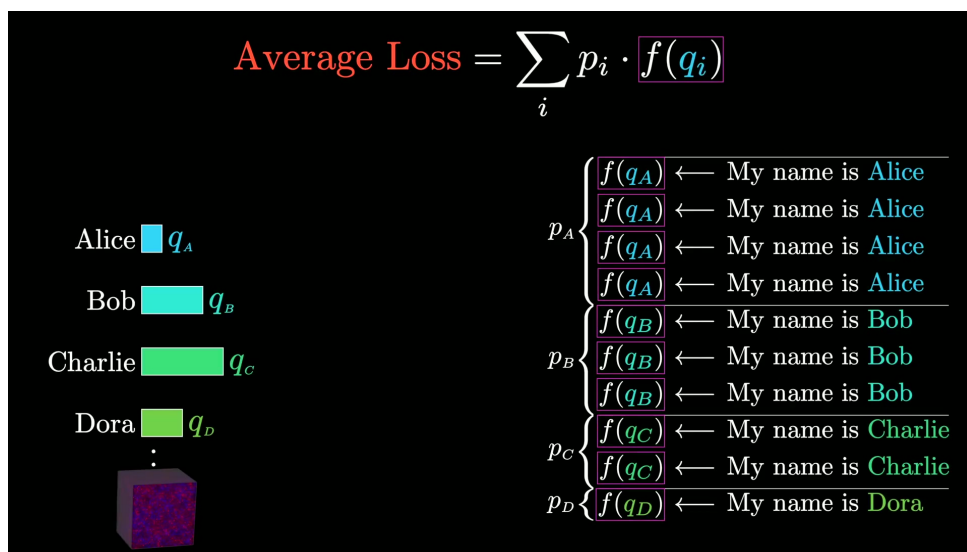


图 19: 平均损失是加权平均 $\sum_i p_i F(q_i)$: 权重 p_i 是名字在数据中的比例。¹⁹

从这里就能预见交叉熵的角色了: 如果选择 $F(q) = -\log q$, 那么这个完整表达式正是交叉熵的定义——具体说, 是模型输出分布相对于数据中统计分布的交叉熵。

而交叉熵的关键性质 ($P = Q$ 时最小) 在这里意味着: 当模型输出的分布与数据中的统计分布一致时, 损失最小。

7.2 反过来: 约束优化强迫你使用对数

但真正关键的点是这个推论可以反过来。如果回到用通用函数 F 写每个示例的损失, 并且要求“所有示例上的平均损失只在模型输出与数据统计一致时才被最小化”——有一个非常漂亮的数学论证说明 F 必须是对数。

3Blue1Brown 把细节 (涉及拉格朗日乘子 Lagrange multiplier) 打在屏幕上给多元微积分爱好者看。高层想法很漂亮: 在 $\sum_i q_i = 1$ 的约束下最小化关键表达式, 问题变成找两条等值线 (contour) 相切的位置; 通过令一对梯度相等来求切点, 对我们的具体表达式会归结为条件: 函数 F 对 q 的导数必须形如“常数除以 q ”——而只有对数有这个性质。

$$\frac{dF}{dq} = \frac{\lambda}{q} \implies F(q) = \lambda \log q + C$$

¹⁹ 视频画面时间区间: 00:23:22–00:23:50。

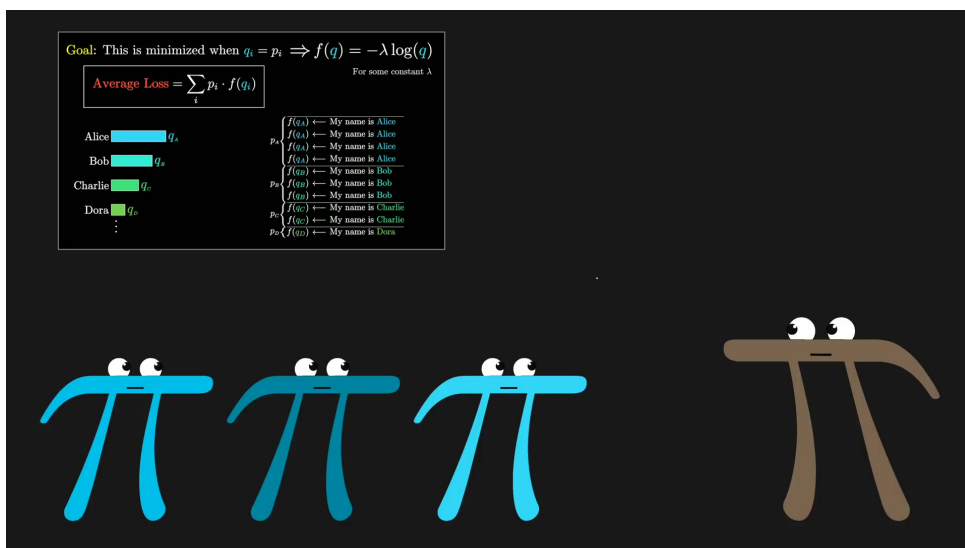


图 20: 约束优化的结论: 目标在 $q_i = p_i$ 时最小化, 迫使 $f(q) = -\lambda \log q$ 。²⁰

手被强迫了

如果你想要一个损失函数, 其性质是”仅在模型匹配数据统计时被最小化”, 你的手就被迫使用负对数。此时, 任何给定输入文本的平均损失看起来就恰好是交叉熵公式。这不是”选择”——是数学的必然。

过拟合的注解

3Blue1Brown 在屏幕上还附了一张关于过拟合的注释: 当你停下来想想我们拥有的数据有多稀疏时, 这套东西居然真的有效, 本身就是一件神奇的事。

8 蒸馏: 交叉熵公式更”显式”的用法

本节主题是语言模型训练的一个有趣变体, 其中交叉熵公式被更显式地使用: 每个训练示例给模型分布一个”更柔和”的拉力, 不那么过度偏向某一个具体 token。它叫做蒸馏 (distillation)。

8.1 设置

你可能想训练一个相对较小的模型, 其目标是逼近某个更大、更聪明的模型的性能。这在实践中太常见了: 通过扩大参数规模获得理想性能后, 你希望用更小、推理时更高效的模型来逼近它。

²⁰视频画面时间区间: 00:24:56–00:25:02。

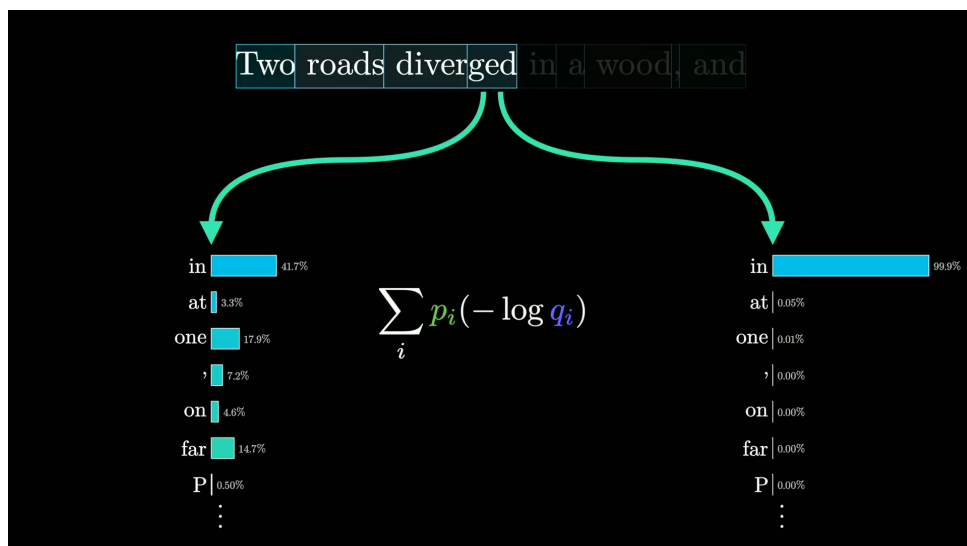


图 21: 蒸馏: 小模型的预测与大数据模型在同一点的完整分布比较, 而不是与数据里的真实下一个 token 比较。²¹

有了大模型在手, 你就能做比普通预训练高效得多的事情: 浏览一堆数据, 像以前一样在每个新 token 处考虑小模型的完整预测分布——但不再与数据中的真实下一个 token 比较, 而是与**大模型在同一点的完整分布**比较。

该 token 处的损失定义为: 小模型分布相对于大模型分布的交叉熵。

蒸馏比预训练”丰富”在哪

普通预训练只把全部权重给唯一真实的下一 token (碰巧在那里的那个)。蒸馏则把大模型的完整分布作为监督信号——它告诉小模型”其他候选 token 也各有各的可能性”。

下棋类比

这就像”只看棋谱学棋”与”高手一边下棋一边给你讲解所有可能的好棋步、以及每一步它们会赋予多大权重”之间的区别。

回到”My name is _____”例子: 普通预训练需要看几千、几百万个示例, 损失才代表对合理名字分布的交叉熵; 而蒸馏只需要**单个示例**——拿它和大模型的好预测比较, 就能达到同样的效果。

9 总结与延伸: 从压缩到智能

9.1 把两条线索织在一起

现在值得退后一步, 审视我们建立的一切:

1. 前半部分 (压缩语境): 交叉熵自然地出现, 只需问”为一种环境优化的压缩方案在另一种环境下表现如何”。轻微惊喜是, 这个想法作为”衡量两种环境模式差异”的工具被证明普适得多——这正是它成为现代机器学习损失函数常用工具的原因。

²¹ 视频画面时间区间: 00:26:53–00:27:20。

2. 后半部分（机器学习语境）：交叉熵的出现方式感觉完全不同——它看起来与压缩无关，逻辑来自数学中完全不同的领域：约束优化。

的类比

就像数学中每当 出现——哪怕看起来与圆无关——背后总是藏着与圆的联系；每当交叉熵出现，也会有办法把它连接到压缩。这是数学在向你眨眼。

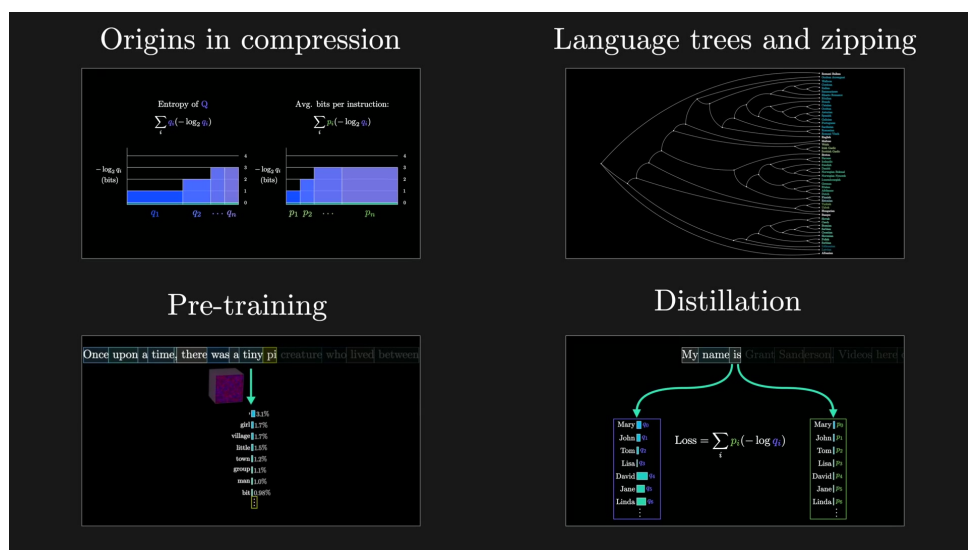


图 22: 回望：交叉熵起源于压缩（语言树与 zipping、熵、平均比特数）。²²

对语言模型来说，我们已经通过”损失 = 模型视角的平均信息量”为连接压缩铺好了路。下一步是解释如何把通用预测器变成压缩器——比如把语言模型变成语言压缩器，其中表示一段文本所用的比特数近似等于该文本从模型视角看的信息内容。这惊人地可行，而且有一个非常直观、美丽的压缩算法（下一部分会深入讲）。一旦理解了它，就会明白：使用交叉熵损失，实际上等价于训练模型成为最好的文本压缩器。

²² 视频画面时间区间：00:28:14–00:28:34。

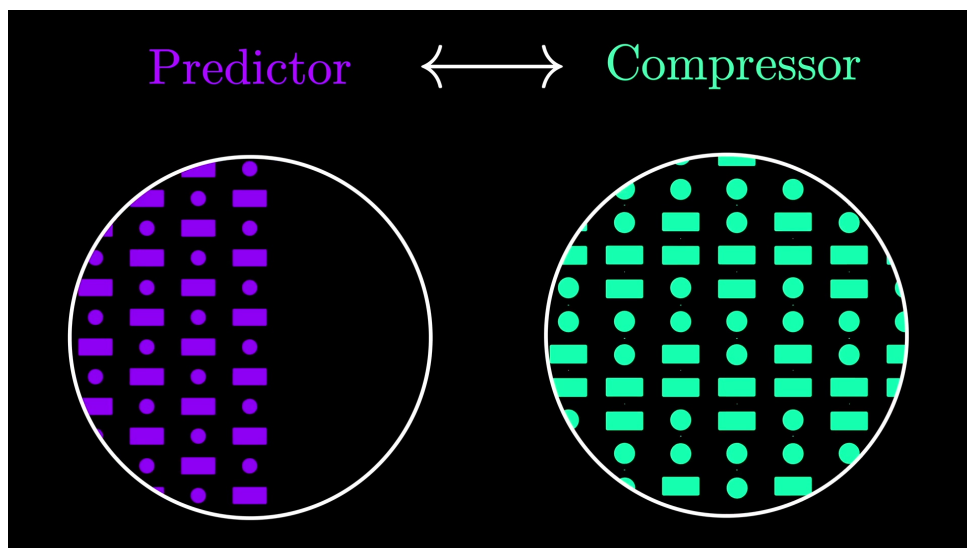


图 23: 通往下一部分: 预测器 $\rightarrow$ 压缩器。²³

压缩即智能

这给了我们一座山丘，站上去可以正面审视那句发人深省的话：**压缩即智能** (Compression is Intelligence)。

9.2 KL 散度：交叉熵 - 熵

视频最后补一个相关概念——KL 散度 (Kullback-Leibler divergence)，机器学习论文里随处可见。

记住交叉熵的关键性质：它在 $P = Q$ 时取最小值，且最小值等于 P 的熵。这自然让人想给“这两个值的差”起个名字——它就是 KL 散度：

$$\text{KL}(P\|Q) = \underbrace{\sum_i p_i(-\log_2 q_i)}_{\text{交叉熵}} - \underbrace{\sum_i p_i(-\log_2 p_i)}_{\text{熵 } H(P)}$$

²³ 视频画面时间区间：00:29:32–00:29:52。

Kullback-Leibler Divergence

$$\left(\sum_i p_i (-\log_2 q_i) \right) - \left(\sum_i p_i (-\log_2 p_i) \right)$$

Cross Entropy of Q relative to P Entropy of P

图 24: KL 散度 = 交叉熵 - 熵 $H(P)$ 。²⁴

在压缩的语言里，它描述的是”由于使用了优化不佳的编码，每个符号浪费了多少比特”。回到机器人例子：未优化情形下每指令比特数减去优化情形下每指令比特数，这个差就是两个分布之间的 KL 散度——它具体告诉你有多浪费。

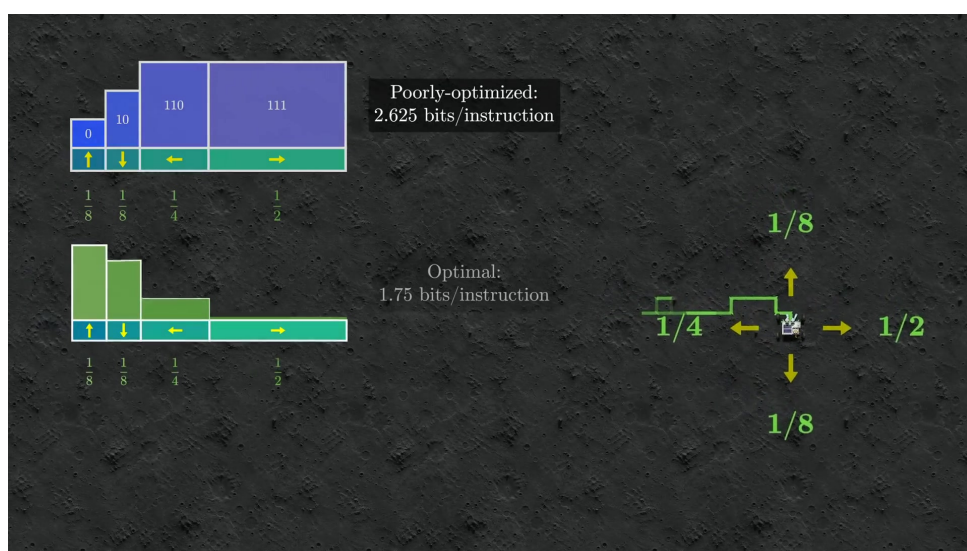


图 25: 机器人例子：未优化 2.625 bits/指令 vs 最优 1.75 bits/指令，差即 KL 散度。²⁵

在机器学习语境中，KL 散度充当分布之间的一种”距离度量”：两者相同时为零，随差异增大而增大。但它不是真正的距离，因为它是**不对称的**： Q 相对于 P 的 KL 散度不等于 P 相对于 Q 的。

²⁴ 视频画面时间区间：00:31:35–00:31:58。

²⁵ 视频画面时间区间：00:32:05–00:32:24。

$$\begin{array}{c} \text{KL Divergence of } \mathbf{Q} \text{ relative to } \mathbf{P}: \\ \left(\sum_i p_i (-\log_2 q_i) \right) - \left(\sum_i p_i (-\log_2 p_i) \right) \end{array} \neq \begin{array}{c} \text{KL Divergence of } \mathbf{P} \text{ relative to } \mathbf{Q}: \\ \left(\sum_i q_i (-\log_2 p_i) \right) - \left(\sum_i q_i (-\log_2 q_i) \right) \end{array}$$

图 26: KL 散度的不对称性: $D_{KL}(Q\|P) \neq D_{KL}(P\|Q)$ 。²⁶

三个思考题（来自视频结尾）

1. 实践中的紧凑形式 $\sum_i p_i \log_2 \frac{p_i}{q_i}$: 验证它与上述展开式等价（它只是隐藏了“交叉熵减熵”的事实）。
2. 视频反复展示的那个 KL 柱状图变体：它在什么意义上表示这个表达式？
3. 蒸馏时用的是两个分布的交叉熵作为损失——如果改用 KL 散度会怎样？毕竟 KL 像距离度量，听起来更自然。（提示：最小化交叉熵和最小化 KL 散度给出的最优 Q 完全相同，因为熵项不依赖 Q 。）

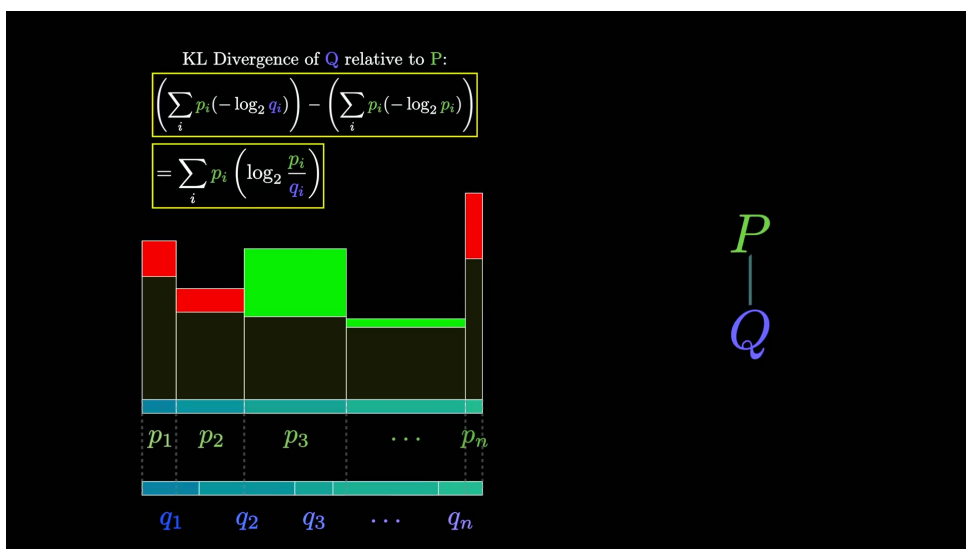


图 27: KL 散度的紧凑形式: $\sum_i p_i \log_2 \frac{p_i}{q_i}$ 。²⁷

²⁶ 视频画面时间区间: 00:32:28–00:32:44。

²⁷ 视频画面时间区间: 00:32:44–00:33:00。

9.3 笔记人的综合思考

概念压缩。整段视频可以用一句话压缩：**交叉熵 = 用“错误世界的码本”给“真实世界”计费。** P 是现实（数据的统计）， Q 是模型（或旧编码）的信念。训练一个模型就是不断调整 Q ，让“账单”最小化；而当 $Q = P$ 时，账单恰好等于熵，即该世界固有的最低通信成本。从这个角度看，损失函数不是任意的“评价工具”，而是可解释的物理量：它测量模型信念与世界现实之间冗余的、浪费的比特。

跨节联系。语言树压缩用 gzip 的“经验交叉熵”恢复语言谱系，与 LLM 用交叉熵损失对齐模型与数据，本质是同一个操作在两个尺度上的展开：微观上衡量“两个文档模式的差异”，宏观上衡量“模型分布与数据分布的差异”。蒸馏则把“数据”这个 P 换成了“大模型的预测分布”——监督信号从稀疏的硬标签变成稠密的软分布。

推广。交叉熵的普适性来自它只依赖“两个分布之间的错配”这一抽象：只要你想让某个可调系统（编码器、模型、小模型）去匹配某个目标环境（真实分布、数据分布、大模型分布），且用期望码长/期望惊讶度计费，交叉熵就是必然出现的代价函数。而约束优化论证更进一步证明：任何“仅在匹配时最优”的损失都必然是负对数——这解释了为什么深度学习中到处是 $\log$ 。

开放问题与下一步。视频留下的悬念是：把语言模型变成真正的压缩器在算法上如何实现（下一部分将用算术编码/类似机制展开），以及“压缩即智能”这个命题能在多大程度上被严格化。对学习者，动手实验建议：用一个小数据集训练一个字符级 n -gram 或小 transformer，观察训练损失如何逼近数据的经验熵；再尝试用大模型的软分布做蒸馏，对比收敛速度。

9.4 拓展阅读

- 本系列第一部分：Compression is Intelligence Part 1——信息内容与熵的建立
- 原论文：Benedetto, Caglioti, Loreto, *Language Trees and Zipping*, Phys. Rev. Lett. 88, 048702 (2002)
- 3Blue1Brown 关于交叉熵的经典视频《But what is a neural network?》系列